Journal of Information Security and Applications 78 (2023) 103608

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

Federated reinforcement learning based intrusion detection system using dynamic attention mechanism

Sreekanth Vadigi [a], Kamalakanta Sethi [b,*], Dinesh Mohanty [a], Shom Prasad Das [c], Padmalochan Bera [a]

[a] Indian Institute of Technology Bhubaneswar, India
[b] Indian Institute of Information Technology Sri City, India
[c] Birla Global University, India

ARTICLE INFO

Keywords:
Intrusion detection systems (IDS)
Federated learning
Dynamic attention mechanism
Reinforcement learning
Deep Q-networks

ABSTRACT

Due to the recent advancements in the Internet of Things (IoT) and cloud computing technologies, the detection and prevention of intrusions in enterprise networks have become a crucial and challenging task. Real-time monitoring of network traffic and resources is required to protect those networks from intrusions. An Intrusion Detection System (IDS) analyses the data packets from the network and the system-level applications to detect any malicious activity. However, existing IDSs require all the data, collected at different network nodes, to be collated at one central location to perform the analysis for any model development. This approach hampers the data privacy at the network nodes as the data needs to be shared with other nodes. Furthermore, many of the existing IDSs are unable to adapt to evolving attack patterns, which may result in poor network vulnerability detection and significant degradation in the performance of the systems. To address these limitations, we present a Federated Deep Reinforcement Learning-based IDS in which multiple agents are deployed on the network in a distributed fashion, and each of these agents runs a Deep Q-Network logic. We considered the data privacy concerns of each agent while designing the system. In our system, the data at each agent node is not shared with any other nodes. At the same time, however, all the agents in the system benefit, via the attention weighted model aggregation process, from the distribution and pattern of the data available at all the other agents. We have also developed an attention mechanism that dynamically determines attention value of an agent, which is used in the model aggregation process. Our model can be scaled to large networks and is resistant to hardware or network failures at any agent node. We have tested and evaluated our proposed system on the cloud-based ISOT-CID dataset and the standard benchmark NSL-KDD dataset. The experimental findings demonstrate the performance and robustness of our proposed model in terms of metrics like accuracy, precision, false-positive rate, and area under the ROC curve.

1. Introduction

The internet has grown in both size and complexity in the last few years along with the evolution of the new kinds of connected devices. As a result of the increasing network complexity, the systems present in the networks are exposed to various security threats, including intrusions. Developing an accurate Intrusion Detection System (IDS) is critical for effectively overcoming various forms of threats. IDS is a key element of network security and acts as the second line of defence for the network. IDS accomplishes this by employing a variety of anomaly detection techniques.

IDS can be divided into two categories based on its position, i.e., Network-based and Host-based IDS. The IDS is positioned on a single system as part of the host-based strategy to monitor its behaviour, including analysis of system calls, information about file system access, application logs, inbound and outbound network traffic, etc. The network-based IDS (NIDS), on the other hand, analyses the traffic and data flow across the network to find any intrusions. Further, network-based IDS are divided into flow-based NIDS and packet-based NIDS. Packet-based NIDS scans each packet that flows through the network, while flow-based NIDS aggregates data from a flow of related packets and detects intrusions. The detection technique used in IDS can also

* Corresponding author.
E-mail addresses: vs25@iitbbs.ac.in (S. Vadigi), kamalakanta.s@iiits.in (K. Sethi), dm22@iitbbs.ac.in (D. Mohanty), shomdas@gmail.com (S.P. Das), plb@iitbbs.ac.in (P. Bera).

https://doi.org/10.1016/j.jisa.2023.103608

Available online 19 September 2023
2214-2126/© 2023 Elsevier Ltd. All rights reserved.

classify them into anomaly-based systems and signature-based systems. Signature-based NIDS has a repository of data related to previously identified attack patterns and detects any network traffic as attack which matches any of the stored attack patterns [1]. However, these systems fail to identify new and unseen attack patterns that are not present in attack repository. Anomaly-based IDS observes any deviation from the regular activity and identifies the attack or intrusion. These systems may be capable of detecting new attack patterns, but their false positive rate (FPR) is high.

In recent years, researchers have developed a large number of ML-based NIDS by employing conventional techniques such as K-Nearest Neighbours (KNN) [2], Support Vector Machine (SVM) [3,4], Decision Trees (DT) [5,6], and Artificial Neural Networks (ANN) [7]. However, these conventional ML-based IDSs give poor performance on large datasets. For performance improvement, deep learning-based models are developed for network intrusion detection. However, the continuous change in attack patterns and the emergence of novel attacks require the NIDS models to retrain frequently. It is also difficult to scale these models to the larger network sizes. Hence the researchers have focused on developing Reinforcement Learning-based intrusion detection systems to overcome such limitations [8]. Reinforcement Learning (RL) is a type of machine learning that processes a series of decision-making tasks to identify the best behaviour for a given situation. It generally includes an agent (which takes actions depending on what it has learned) and an environment that gives rewards and punishments based on the agent's expected behaviour and action. The idea of RL is influenced by way in which human beings learn a task, which is to see the results of a series of trials until they master it. With no or minimal prior information about the environment, Reinforcement Learning can model an agent to conduct subsequent actions effectively. Due to its high adaptability and effectiveness in standard and adversarial scenarios, it is well suited for the network security domain, as the cyberattacks are getting more complex and are continuously evolving. Reinforcement Learning utilizes the total capacity of Artificial Intelligence to identify anomalies. However, conventional RL has some limitations, such as scalability and the inability to develop advanced security models. Deep Reinforcement Learning (DRL) [9] is a newer research area that can help to design complex models which can identify extremely sophisticated cyberattacks [10]. This idea has found excellent applications in computer vision, healthcare, and robotics [11]. DRL is also an emerging topic in the domain of network security, especially in next-generation IDS research and development.

Data privacy has recently become a significant concern for both individuals and organizations. Traditional Intrusion Detection Systems, that employ ML methodologies, need the data to be gathered in one centralized location in order to train the model. This technique results in compromising data privacy at the end systems/networks. We propose a Federated Deep Reinforcement Learning-based Intrusion Detection System (FDRL-IDS) to address this problem. Our proposed IDS does not require the end agents to share their data with any other agent in the network, and subsequently, it learns from the attack patterns that are observed at each of these agents.

The major contributions of our proposed IDS are mentioned below.

1. ***Federated RL-based IDS***: In the present work, we propose a FDRL-IDS which employs a deep Q-network at each of the agents which are distributed in nature.
2. ***Data privacy***: We took data privacy at each of the agents into consideration when designing and implementing the model. The data at each of the agent node will be processed at that node itself and won't be shared to any other agent. In addition, we use the learning of each of the agent to develop a global model by aggregating these individual agent networks. In this approach, we protect the data privacy at each of the agents and at the same time we can develop a global model by utilizing the data patterns present at all the agents.
3. ***Dynamic attention value mechanism***:- We developed a dynamic attention value mechanism that determines the attention value for each of the available agents in the system. The attention value is determined based on the aggregated model performance on the distribution of the data at that particular agent. If the performance for a particular agent needs to be improved, then its attention value is higher to improves its priority during the model aggregation process.
4. ***Scalability***:- Our proposed IDS architecture can be scaled easily to networks of any size. The attention mechanism and model aggregation process is designed and developed in a way that it allows the dynamic nature of the network such as dynamic addition and removal of network routers or hosts on which agents are deployed.
5. ***Robustness***:- The system is designed to be more resilient to single points of failure, meaning that even if one or more agents in the network got disconnected due to hardware or network failure, it doesn't affect the entire system and the rest of the agents will work without any disruption.

We implemented the proposed system using PyTorch library [12] in Python and evaluated the potential of the system on the datasets ISOT-CID [13] and NSL-KDD [14].

The rest of the paper is organized as follows. Section 2 introduces the background information and basic knowledge on the topics such as federated learning, reinforcement learning and the attention mechanism. Section 3 discusses about the related work. Section 4 presents a comprehensive explanation of our proposed IDS algorithm. Section 5 discusses about the datasets we used in the experimentation and their preprocessing. Section 6 demonstrates our conducted experiments and their results. Shortcomings of our proposed model are discussed in Section 7. Finally the conclusion of the paper is presented in Section 8.

2. Background

This section presents the fundamental knowledge about federated learning, attention value mechanism and the reinforcement learning.

2.1. Federated learning

Traditional Machine Learning algorithms require a single data centre to store the training data. However, it is not advisable to store the data at one centralized location in many real-life applications due to concerns like data breaches and compromised user privacy [15]. Federated Learning [16], a collaborative learning approach, is helpful in such situations, which trains a centralized model without sharing the user's raw data and reduces the load on the communication network. It is inspired by techniques from multiple domains such as Machine Learning, distributed systems, communication architecture, and user data security/privacy.

Multiple users/agents collaboratively train ML models in a Federated Learning environment without exchanging any local raw data generated or collected by the individual agents. In this system, all the agents train their respective models locally and periodically communicate with the central server to train a global model (by aggregation of the local model weights). The trained global model weights are then sent back to all the available agents. The local model weights are then updated with the received global model weights.

2.2. Reinforcement learning

Reinforcement learning is an advanced machine learning approach that intends to train an agent how it should behave in a particular environment. The agent perform actions and receives the feedback from the environment to subsequently improve its behaviour. The major

requirement of solving a reinforcement learning problem is to find an optimal policy mapping from state to action space that maximizes the total future reward. Usually a reinforcement problem can be modelled as a Markov Decision Process (MDP) [17] which consists of five components, i.e., state, action, reward, policy and value. The policy of an agent defines how it behaves in a particular state. This can be accomplished in a variety of techniques, from a simple lookup table to a complex Q-network. Reinforcement learning's major goal is to find out the best policy for the agent. Reward is the response of the environment to the agent's action. In order to obtain the maximum possible total reward, the agent updates its policy function based on the reward it receives from the environment. The value function predicts the agent's total future rewards and is used to determine the value of a state and decide the next course of action. Suppose, the agent begins in state s. Then, the value of a state $V(s)$ is the expected reward which is expressed as

$$V(s) = E(G_t|S_t = s) \tag{1}$$

where G_t is the total rewards starting from time t. It can be defined as the sum of immediate reward and the discounted future rewards. Mathematically, $G(t)$ is given by

$$G(t) = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{m=0}^{\infty} \gamma^m R_{t+m+1} \tag{2}$$

The discount rate γ represents the degradation factor of the future rewards. It lies between 0 and 1. At each time step t, the agent performs an action A_t based on the current state S_t, then the environment provides the feedback in the form of reward R_{t+1}, discount γ_{t+1}, and the next state S_{t+1} based on the performed action. This process of intercommunication is called as the Markov Decision Process (MDP).

2.2.1. Q-learning

Q Learning is a reinforcement learning algorithm which uses Q (stands for Quality) function to estimate reward values. For any MDP, Q-learning finds the best action selection policy with the intention of obtaining the maximum expected value of the total reward [18]. In Q Learning, temporal difference learning is used which is a prediction of the expected future rewards. A smaller value of temporal difference indicates that the agent has adapted to the environment very well. Therefore, the main objective of the Q-Learning algorithm is to minimize the temporal difference values.

The Equation for updating Q value is stated as follows:-

$$Q^{new}(s_t, a_t) \longleftarrow Q(s_t, a_t) + \alpha * (r_t + \gamma * max_a Q(s_{t+1}, a) - Q(s_t, a_t)) \tag{3}$$

where, s_t, a_t, r_t, α, γ, $Q(s_t, a_t)$, $max_a Q(s_{t+1}, a)$ are the state, action taken, reward obtained at time t, learning rate, discount factor, old Q value, and the estimate of optimal future value respectively and $[r_t + \gamma * max_a Q(s_{t+1}, a)]$ is the TD target whereas $[r_t + \gamma * max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$ is the TD equation.

2.2.2. Deep Q-learning

A major drawback of Q-learning is that it works only in the environments that have discrete and finite state–action spaces. In order to extend Q-learning to complex environments, we use Deep Neural Networks as approximating functions that can learn the value function by taking the states as inputs. Here, we refer complex environment as systems where size of state–action table is very large and thereby, it is infeasible to store this table. For complex environments, instead of using Q Learning, we can apply Deep Q-Learning methodology. In this approach, the Q values for all possible actions space for a state is estimated with the help of Deep Neural Networks. The loss function of Deep Q-Network (DQN) is given by

$$loss = (r + \gamma * max_{a'} \hat{Q}(s, a') - Q(s, a))^2 \tag{4}$$

where s, a, r, γ, $\hat{Q}(s, a')$ are the state, action taken, reward obtained, discount factor and the delayed Q function respectively. We can notice the similarity between the loss function of the Deep Q-Network and the temporal difference of basic Q-Learning approach.

2.2.3. Prioritized experience replay

Many RL models stores the previous samples in replay buffer and retrain the model with those samples to improve the agents in remembering and reusing previous experiences. The samples are picked at random in the basic experience replay technique. However, in [19], authors highlighted that picking samples based on their ability to increase the learning of RL agent would result in far superior learning from previous experiences. Therefore the notion of prioritized experience replay comes into play, in which samples are prioritized based on metrics similar to TD.

In particular, the priority for ith sample is given by $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$, where $p_i = |\delta_i| + \varepsilon$. Here, α is the prioritization factor and δ_i the TD error function on which the Huber Loss Function [20] is applied. Mathematically, δ_i is given by

$$\delta_i = r_t + \gamma Q(s_{t+1}, argmaxQ(s_{t+1}, a; \theta_t); \theta_t^-) - Q(s_t, a_t; \theta_t) \tag{5}$$

Sometimes training with a given sample multiple times may introduce biasing of agent. Such biasing is avoided with the help of Importance Sampling. In Importance Sampling, weight given by $w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta$ is multiplied to the TD value while training the agent with ith sample. Here N is the number of samples in the memory and β is a tuning factor. This process of RL model retraining with samples picked based on their respective priorities have shown faster learning of the agents and better performance when tested on the Atari Dataset.

2.3. Attention mechanism

Attention mechanism is one of the most important concept used broadly in the domain of deep learning research, computer vision and natural language processing (NLP). In NLP domain, attention mechanism has shown excellent results and used to improve the performance of encoder–decoder architecture in the field of machine translation. The methodology was designed in such a way that the decoder could use the most important parts of the input sequence. This was done by a weighted combination of all the encoded input vectors where the weights are proportional to the importance of that particular vector. As per the work in [21], an attention functions can be defined as mapping of a query and a set of key–value pairs to an output.

2.3.1. General attention

Several researchers have worked on various forms of attention mechanisms. The paper [22] employs "dot product attention", in which the value of attention is decided by the dot product of weights and values. The authors of [21] used "scaled dot product attention", in which the attention values are scaled down using a scaling factor. The researchers in [23] have worked on "additive" attention that applies *tanh* function on the dot product.

2.3.2. Self attention

Self attention is a form of attention mechanism that computes a representation of a sequence by linking various locations in that sequence. Self-attention can also be referred as intra-attention. In language translation, the paper [24] employs a self-attention mechanism to understand the relationship between current words and the prior part of a sentence.

3. Related work

In this section, we discuss the prior works on the IDS. We majorly concentrate on the research works on IDS that use machine learning, deep learning, reinforcement learning, and federated learning approaches.

In previous IDS works, researchers employed several ML techniques such as K-Nearest Neighbours (KNN) [2], Decision Trees [5,6], Support Vector Machines (SVM) [3,4], Clustering techniques [32,33], Random

Table 1
Comparison of existing IDS models based on ML algorithms.

Ref.	Algorithm	Dataset	Accuracy	Limitation
[25]	DT	NSL KDD	90.31%	High space complexity.
[26]	SVM	UNSW-NB15	85.99%	Not suitable for large datasets and difficult to select optimal parameters
[27]	NAV	Kyto Dataset	98.60%	Algorithm assumes that all the features in the data are independent of others, but this happens rarely in real life scenarios
[28]	ANN	Self Generated	97.51%	Computational complexity is high compared to other traditional Machine learning algorithms
[29]	CNN	CICIDS2017	99.56%	High computational complexity
[30]	RNN	CICIDS2017	96.24%	Vanishing/Exploding gradient.
[31]	DQN	NSL KDD	78%	Data privacy (All the data needs to be at a single place)

DT: Decision Tree, SVM: Support Vector Machine, NAV: Naive Bayes, ANN: Artificial Neural Network, CNN: Convolutional Neural Network, RNN: Recurrent Neural Network, DQN: Deep Q-Network, FL: Federated Learning, RL: Reinforcement Learning, FRLD: Federated Reinforcement Learning with dynamic attention.

Forest [34] and Artificial Neural Network (ANN) [7]. Those IDS models make the system more complex and less efficient in scenarios where there is high volume of heterogeneous data. Injadat et al. [35] proposed an IDS using traditional ML techniques like SVM, RF and K-NN. They have used Bayesian techniques to optimize hyper-parameters of Support Vector Machine, Random Forest and KNN algorithms. They used ISCX 2012 dataset to analyse the performance of the model. In paper [36], authors presented a machine learning-based hybrid intrusion detection approach and proposed an unique feature selection method to choose optimal features. The IDS model performance was assessed using the NSL-KDD dataset. For binary class and multi-class, the model has an accuracy of 99.5% and 98.56%, respectively.

Due to the capacity to capture data representations at diverse labels, deep learning (DL) based approaches have been extensively used in recent years for creating effective and trustworthy IDS [37]. Ayyaz-ul-Haq et al. [38] developed a recurrent neural network (RNN) based Intrusion Detection System for the IoT (Internet-of-Things) application. The proposed IDS performance is assessed using the standard NSL-KDD dataset. When compared to traditional Machine Learning approaches, experimental results revealed that the proposed IDS is more accurate. Jiang et al. [39] presented a deep learning IDS using the long short term memory RNN (LSTM-RNN) [24]. Hassan et al. [40] designed a Deep Learning based IDS to detect network intrusions in big data environment. The proposed IDS model combines Convolutional neural network (CNN) with a weight-dropped, long short-term memory (WDLSTM) network. CNN is used to extract important features and WDLSTM is used to learn dependency among important features to prevent over fitting problem. They evaluated the performance of the model using two publicly available datasets (i.e., UNSW-NB15 and ISCX2012). Wang et al. [41] designed an integrated DL based model to overcome long training time and low classification accuracy of existing DL based IDS models. This IDS model is based on SDAE-ELM and DBM softmax model. This model outperforms over several ML models when evaluated with public datasets (like NSL-KDD, UNSW-NB15).

Recently, researchers have begun to use reinforcement learning in the development of complex IDSs. Cannady delivered one of the early research papers on the design of reinforcement learning (RL) based IDS [42]. To automatically identify new intrusions, he used CMAC (Cerebellar Model Articulation Controller) in the development of Intrusion Detection System. The research work in [43] emphasizes the significant challenges in implementing reinforcement learning based IDS. The paper also discusses about Q-learning based multi-agent reinforcement learning system. Xu and Xie [44] presented a novel method to IDS model development that transforms the intrusion detection problem to a task of predicting the value function of Markov reward process. Alavizadeh et al. [31] proposed Intrusion Detection System using deep reinforcement learning model to detect intrusions in the network. This model has self learning approach that allows the model to improve its intrusion detection ability continuously. The proposed model is very successful in detecting network intrusions and performs superior in comparison to other machine learning approach.

Recent research studies have begun to work in the direction of federated learning (FL) [45] approach for the development of privacy preserving IDS. In paper [46], the authors have presented a review on the use of federated learning in intrusion detection system and discussed various challenges in the implementation of FL-based IDS. The researchers in [47] have proposed a novel federated deep reinforcement learning approach to build models of high quality for the agents and evaluated their framework in the domains of Grid-world and Text2Action. The research work in [48] used federated deep reinforcement learning in the IoT domain. In [49], the researchers have used attention weighted federated deep reinforcement learning (AWF-DRL) for the application of device to device assisted heterogeneous collaborative edge caching. In Table 1, we have given an overview of performance and limitations of various existing IDS models designed using different machine learning algorithms like Decision Tree, SVM, Naive Bayes, Artificial Neural Network (ANN), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN) and Deep Q-Network (Reinforcement Learning).

However, none of the current works have implemented the concepts of federated learning approach combined with the deep reinforcement learning methodology for the IDS applications. In this paper, we present an advanced federated learning based IDS with deep reinforcement learning techniques that adapts to the continuous change in attack patterns. We have also designed and incorporated a dynamic attention value mechanism to improve the performance of the overall system. Our proposed system is scalable, fault tolerant, robust and able to achieve the performance metrics comparable to the state-of-the-art works.

4. Proposed intrusion detection system

This section presents our proposed Federated learning based IDS and its architecture. We also explain about our dynamic attention weightage based distributed reinforcement learning approach for the detection of network intrusions.

The devices present in a computer network can be broadly classified into two types, i.e., routers and hosts. In a network, the data transfers from one host to another through routers. Hosts are more vulnerable to cyber attacks than routers since they act as both sources and destinations for traffic flow. Hence routers are a better choice for incorporating IDS compared to host machines. In our proposed architecture, the routers are equipped with IDS which runs deep reinforcement learning models. We refer to these detection models as agents, each of which collect the essential data required for the intrusion detection from the network packets and obtain feature vectors, which are then sent to the model for intrusion detection.

In addition to the agents, the system will also contain a central server for receiving and aggregating the distributed agent models and then sending the aggregated model weights to all the available agents who would then update their respective local models. The Fig. 1 displays the architecture of our proposed distributed reinforcement learning based Intrusion Detection System.

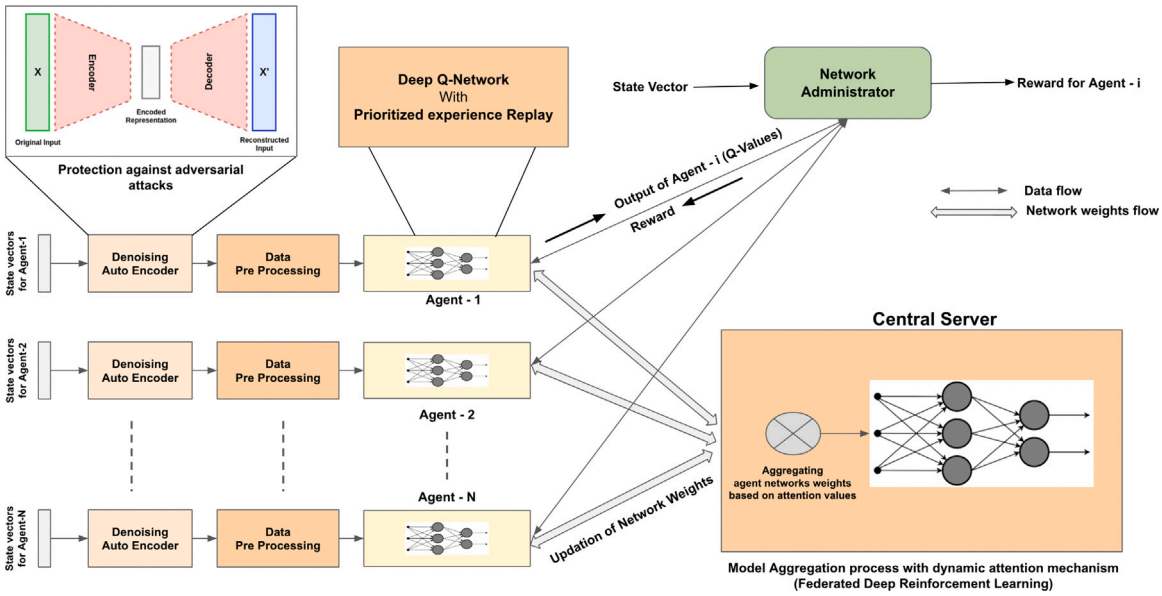


Fig. 1. Overview of the proposed system architecture.

4.1. Agent algorithm

All the agents are equipped with a Deep-Q network for employing the reinforcement learning approach to detect the network intrusions. Each agent receives packets from the network traffic, and uses them to extract feature representations. These features representations will be passed through a denoising autoencoder (DAE) to protect the model from the adversarial attacks. The output of the DAE is then passed to the data preprocessor to normalize the learned state representation. The preprocessed state vector will then be provided as the input to the agent's Deep Q-Network and the Q values get generated. Based on these generated Q values and the required output, a reward r is awarded to the model, using which the weights of the Q network gets updated. The detailed step by step procedure followed by the agents is presented in Algorithm 1 and is explained in the following paragraphs.

During each round of training process, the agent is given a batch of training samples. For each of the feature vectors, the current vector is set as the current state and fed as input to the agent's Deep Q-Network. Based on the generated Q_value_vector from the DQN, we predict the action (whether the given state is an attack/non-attack). The agent chooses the action for which the Q value is the highest. If action 0 has maximum Q value, it indicates that the agent predicts the given state or the current sample belongs to the non malicious category. Similarly if action 1 has maximum Q values indicates that the agent predicts the given state belongs to an attack/malicious category. Based on the original category of the sample from the dataset, the agent obtains a reward r and the $target_Q_value_vector$. Then the agent updates the weights of its Deep Q-Network based on the reward obtained using the Bellman's equation.

An error vector is calculated by the absolute difference between the elements of the predicted Q_value_vector and the $target_Q_value_vector$. This error vector is used in calculating the weight ($current_state_loss_weight$) of the experience for the purpose of prioritized experience replay [19]. This weight is calculated as the summation of the elements in the error vector and raised to the power ω, a hyperparameter in prioritized experience replay. A memory M is allocated at the beginning to cache the experiences for the prioritized experience replay. Then the experience tuple along with the state loss weight is stored in the memory. These iterations lead to the training of the agent's DQN. Then a batch of experiences are sampled from the memory where the probability of selecting an experience is directly proportional to its loss weight which was calculated earlier. The training cycle is repeated on the sampled batch to ensure better learning of the DQN from the data points from which the agent needs to learn more.

Simultaneously, during the execution of the above process at each of the distributed agents, the central server runs Algorithm 2 and the attention values for each of the agents will be computed dynamically using Algorithm 3.

In Algorithm 2, the central server will be provided with the network weights of all the available agents along with their attention values. After obtaining the input, it will compute the W_{sum}, sum of network weight matrices which are multiplied by their respective attention values. Also it will calculate the summation of all the attention values i.e., $total_attention_sum\ A_T$. The previously computed W_{sum} is then divided by the $total_attention_sum\ A_T$ to find the aggregated network weights (W_{agg}). This can be described as the weighted average of the available Q-networks which are weighted on their respective attention values. The computed W_{agg} will then be sent to all the agents, which then replace their respective local Q-networks with these aggregated weights and continue their training process.

In Algorithm 3, we mentioned the process how agents dynamically determine its attention value based its training set and the performance of the aggregated model on the test dataset. It is noted that test dataset is a constant subset sampled out from the available training data to represent its data distribution at the agent. Each agent initializes a variable $attention_multiplier$ which is set to 1 at the beginning of the training process. After each round of network weight aggregation, the model receives the aggregated network weights from the central server and updates/replaces its local model with these aggregated weights and computes the performance metrics (eg: accuracy) on the test dataset. Based on the computed accuracy and the number of samples used in the current round of training process, we will update the $attention_value$. Here, attention value is calculated by multiplying $num_samples$ by an $attention_multiplier$. Attention multiplier is a function of the performance metrics (we chose accuracy in our experiment) that determines the additional weight that needs to be provided to the current agent. It is used to improve the performance of the aggregated model on the data distribution at the current agent. Mathematically, attention multiplier is given by

$$attention_multiplier = 1 + k * (1 - accuracy) * (a^{-accuracy}) \tag{6}$$

Algorithm 1: Core algorithm for agents

$X_1, X_2, \ldots X_N$ - Feature vectors are provided as input to the agent;
Set the value of epsilon ε - the hyperparameter of epsilon-greedy approach;
Allocate the memory M required for prioritized experience replay;
Assign the appropriate value of E, which is the count of total episodes;
for *i in range 1 to E* **do**
  Shuffle all the available feature vectors to randomize the sequence of states.
  for *j in range 1 to N* **do**
    Set $state_vector \leftarrow X_j$;
    Send the current $state_vector$ as the input to agent's DQN and collect the output values as Q_value_vector;
    Select action based on the Q_value_vector, $action$ = argmax(Q values of the two actions) with the epsilon-greedy methodology;
    Based on the actual output from the dataset and the predicted action, obtain the reward r;
    Update DQN weights using Bellman's equation: $Q_{\text{new}}(s,a,r) = r + \gamma * Q_{\text{old}}(s,a,r)$;
    Calculate the $error_vector = |Q_value_vector - target_Q_value_vector|$ where the $target_value_vector$ contains 0s at all indices except at the actual action's index (contains 1);
    Compute the $current_state_loss_weight = (\sum_{i=1}^{i=n} error_vector[i])^{\omega}$, where n is the size of $error_vector$;
    Store the tuple $< current\ \ state, action\ \ taken, reward, current_state_loss_weight >$ in memory M;
  end
  Select a sample batch of size b from the memory M, where the probability of selecting each sample, i.e., P_s is proportional to the $state_loss_weight$ obtained during its training phase;
  Replay the training process on the sampled batch;
end

Algorithm 2: Central server algorithm

Input : Deep Q-Network weights for each of the available agents: $W_1, W_2, \ldots W_N$ and Attention values for each of them: $A_1, A_2, \ldots A_N$;
Calculate the sum of attentions values of all the available agents. Total attention sum $A_T = \sum_{i=1}^{i=N} A_i$;
Initialize a weight matrix W_{sum} with zeros in the shape of Q-Network weights;
for *each agent k* **do**
  $W_{sum} = W_{sum} + A_k * W_k$; where A_k and W_k are the attention value and the weight of the kth agent respectively.
end
Divide the calculated weighted sum by the total attention sum to obtain the aggregated network weights, i.e., $W_{agg} = W_{sum} / A_T$;
Send the computed W_{agg} to all the participating agents and each agent will then update its own Q-network with this aggregated network weight matrix and continue its training process

Algorithm 3: Dynamic Attention value computation

Input : Test dataset representing data distribution at the current agent and the aggregated model weights received from the central server
Initialize $attention_multiplier = 1$;
Compute the $accuracy$ of the aggregated model on the available test dataset;
Calculate $attention_multiplier = 1 + k * (1\text{-}accuracy) * (a^{-accuracy})$, where k and a are the hyperparameters;
After each round of the training for all the agents, obtain the number of samples = $num_samples$ used during the training process, and compute the attention value using the equation $attention_value = num_samples * attention_multiplier$, and send it to the central server along with the local network weights for model aggregation.

In the above equation, k and a are the parameters of the function which can be fine tuned for obtaining the best aggregated model that performs well on different data distributions at all the agents. The derivation of attention multiplier equation is given in Section 4.4.

4.2. Purpose of attention value

In some cases, the training data present at one agent is much less when compared to the data present at the other nodes. For example, let us assume that there are two agents A1 and A2 and the number of samples used in the training process of A1 are twice the number of training samples in A2, in each round. In this case, we need to give more weightage to the network of A1 during the aggregation process since it is trained on a large amount of data. Here the concept of attention value comes into the picture. Based on the weightage that needs to be given to the agent, its attention value is determined. i.e., $attention_value$ is directly proportional to the weightage required. A simple formulation for the attention value is to use the number of samples used in the current round of training process at the agent, i.e.,

$$static_attention_value = number_of_training_samples \tag{7}$$

4.3. Purpose of dynamic attention computation

In some cases, it might be possible that an agent is getting trained on a lesser number of samples, but the type/distribution of the data present at that agent is much more important. Consider, for instance, that there are two agents A1 and A2 in the intrusion detection system and the data given to A1 contains a specific types of attacks while data given to A2 contains different types of attacks. Size of data at A1 is ten times larger than that at A2, but the type of attack in the A2 dataset is much more severe although the volume of packets involved is lesser. In this scenario, giving attention value to the agents just based on the number of samples/dataset size doesn't give us promising performance after model aggregation. This happens since the agent that trains on the severe attack type is not given enough importance (attention) during the model aggregation because of its smaller dataset,

To overcome the above specified limitation, we derived a new mathematical model to dynamically determine the attention value for each agent. This approach considers both the dataset size and the performance of the aggregated model on the data distribution at each

agent. We used accuracy as a performance metric for the model and came up with an equation for attention value which is shown below

$$dynamic_attention_value = num_training_samples * attention_multiplier \quad (8)$$

By substituting the value of *attention_multiplier* from Eq. (6), we get

$$dynamic_attention_value = num_training_samples * (1 + k * (1 - accuracy) * (a^{-accuracy})) \quad (9)$$

If we consider Eq. (6), attention multiplier is a function of accuracy, and the parameters k and a. These parameters play an important role in determining the attention values for the agents. Since the accuracy would always lie in between 0 and 1, the minimum value of the attention multiplier is 1 and the maximum value would be $1 + k$. As the accuracy model goes from 0 to 1, the value of attention multiplier would drop from $1 + k$ to 1. The value of k determines how high the attention value should be and the value of a determines how fast the attention multiplier drops as the accuracy moves from 0 to 1. The attention multiplier increases as we increase the value of k, and it decreases more quickly as we increase the value of a. Fig. 2 show the behaviour of the attention multiplier based on the parameters k and a.

4.4. Derivation of attention multiplier equation

As mentioned earlier, we require a dynamic attention mechanism to ensure good performance of the model during cases when the data across the agents is not evenly distributed. So we need an equation to determine the dynamic attention value.

The attention value should be high when the agent's accuracy is low (initially at the beginning of the training process) and should gradually reduce to 1 as the agent's accuracy reaches 1. Let k denote the value of the attention multiplier when the agent's accuracy is 0. It is also the highest possible attention multiplier value that an agent can have. So we need an equation that passes through the points (0, k) and (1, 1). We initially begin with basic quadratic (10) and exponential (11) equations that pass through the required points.

$$y = (k - 1)x^2 - 2(k - 1)x + k \quad (10)$$

$$y = k^{1-x} \quad (11)$$

But with the Eqs. (10), (11) we realize that we cannot control the speed or the slope with which the value falls from k to 1 as the accuracy goes from 0 to 1. So we come up with Eq. (12) and later modify it to Eq. (13) with which we can control the slope of the curve with the help of additional parameter a.

$$y = 1 + k.e^{-x} \quad (12)$$

$$y = 1 + k.a^{-x} \quad (13)$$

But the Eqs. (12), (13) are not passing through the point (1,1). So we modify Eq. (13) to Eq. (14), whose value falls as x moves from 0 to 1 and also passes through (1,1) and (0, k'), where $k' = 1 + k$. In this equation, the parameter k controls the highest value of the attention multiplier, and the parameter a controls the speed or slope with which the value drops from k' to 1. We used the desmos website [click here] to check how the equation curve varies by changing the parameters k and a. The parameters k and a can be fine-tuned to get the best performance.

$$y = 1 + k.(1 - x).a^{-x} \quad (14)$$

By substituting x with *accuracy* and y with *attention_multiplier*, we get Eq. (6), which we have used in our dynamic attention mechanism

$$attention_multiplier = 1 + k * (1 - accuracy) * (a^{-accuracy}) \quad (15)$$

Table 2
NSL-KDD Dataset.

Label	Training	Testing	Total
Normal	125 973	22 544	148 517
DoS	45 927	7458	53 385
Probe	11 656	2421	14 077
R2L	995	2887	3882
U2R	52	67	119

5. Dataset description and preprocessing

We have used the standard NSL-KDD Dataset and the cloud based ISOT-CID for the evaluation of our proposed federated deep reinforcement learning based IDS. In this section, we will provide a brief description of both the datasets.

5.1. NSL-KDD Dataset

NSL-KDD [14,50] is a well-known benchmark datasets, used to evaluate the performance of any IDS. This dataset was developed in 2009 by Tavallaee et al. [51] based on the KDDCUP'99 dataset. There are 148 517 records in the NSL-KDD dataset including both the classes i.e., malicious and non-malicious. Each record has 43 attributes of which 41 features are related to network traffic and the last two represents the label (normal/attack) and the severity of input record. Every label in the dataset is either normal or one of the 38 types of attacks. The training set includes only 22 attack types, where as the testing set contains additional 16 types of attacks. All these classes can be categorized into 4 groups: DoS, Probe, R2L (Remote to local) and U2R (User to Root). Out of all the 41 attributes, there are 4 categorical, 6 binary, 10 continuous and 23 discrete features. The distribution of the number of samples in each class present in the dataset is shown in the Table 2.

5.2. ISOT-CID Dataset

ISOT-CID [13] is the first publicly available intrusion dataset which is captured in a real world cloud environment. The data was collected at various layers (such as network layer, guest host layer and the hypervisor layer) of the hypervisor nodes in the OpenStack based cloud environment [52]. The data was collected in two phases and we have used the dataset collected in second phase since it happened more recently and covers new attack patterns. The dataset covers various attack patterns like simultaneous attacks, coordinated attacks and also the attacks that were launched from multiple continents across the globe. The non-malicious records present in the dataset covers wide variety and complex scenarios with the usage patterns of 160 authorized users involving the data from VM maintenance status, creation of files, SSH, updating and rebooting etc.

The attack patterns in the ISOT-CID can be classified into insider and outsider attacks based on the user who performed the attack (insider/outsider). Insider attacks occurs due to an internal user who has high privileges on the cloud systems or due to a compromised VM. A compromised VM may in the future be used to target other instances that are present in the cloud environment. On the other hand, the outsiders attacks refers to the attacks that are performed from the outside world.

The ISOT-CID dataset is of 8 terabytes in size, out of which the network traffic data comprises 55.2 GB. This data was collected at various levels of network communication: internal traffic, external traffic and local traffic. Internal traffic refers to the communication between the hypervisor nodes. External traffic refers to the communication between different instances whereas local traffic refers to the communication between different VMs on the same hypervisor node. The data related to network traffic was collected in the form of packet

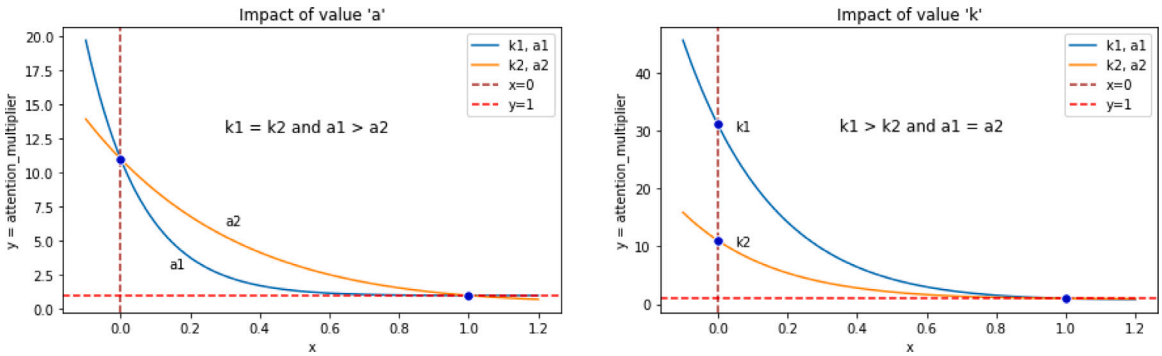


Fig. 2. Graph of the attention multiplier.

capture (PCAP) format. In the first phase, 22,372,418 packets were captured out of which 0.07% (15,649) are malicious. Further in the second phase 11,509,254 packets were collected and among them 17.43% (2,006,382) are attacks.

6. Experimental results and discussion

The performance of the proposed architecture is tested and evaluated on two datasets, i.e., standard NSL-KDD dataset and the cloud based ISOT-CID dataset. We have conducted two types of experiments on both the datasets. In the first experiment, all the available data is randomly split into approximately equal parts and shared among the agents. On the other hand, in second experiment, each agent is provided with a customized data distribution, such as splitting the dataset into multiple parts wherein each part has a certain type of attack types and each set varies in number of samples when compared to other sets. For the first experiment (random splitting and distributing among agents), we simulated an IDS containing eight agents and one central server.

Standard machine learning metrics are used to evaluate the model performance of our proposed system, such as accuracy, precision, F1-Score, recall, Area under ROC curve (AUC) and false positive rate (FPR). These metrics are derived from the values: True Positive, True Negative, False Positive and False Negative.

- TP: True Positive is the number of outcomes in which the model correctly predicts the positive class.
- TN: True Negative is the number of outcomes in which the model correctly predicts the negative class.
- FP: False Positive is the number of outcomes in which the model incorrectly predicts the positive class.
- FN: False Negative is the number of outcomes in which the model incorrectly predicts the negative class.

Different metrics can be computed from the above terms as follows:

- **Accuracy**: It is the ratio of the total number of correct predictions to the number of total predictions.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

- **Precision**: It is the ratio of the number of correctly predicted positive samples to the number of samples that are predicted as positive.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall**: It is the ratio of the number of correctly predicted positive predictions to the number of samples that are actually positive.

$$Recall = \frac{TP}{TP + FN}$$

- **False Positive Rate**: It is the ratio of the number of false positives to the number of samples that are actually negative.

$$FPR = \frac{FP}{FP + TN}$$

- **F1-Score**: It is the harmonic mean of recall and precision.

$$F1Score = \frac{2 * Precision * Recall}{Precision + Recall}$$

- **Receiver Operating Characteristic (ROC) Curve**: It is a curve that is drawn with false positive rate (FPR) on x-axis and true positive rate (TPR) on the y-axis. The probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance is represented by the area under the ROC curve (AUC). The ability of a model to distinguish between multiple class labels is measured by its AUC. The higher the AUC score, the better the model distinguishes between the various classifications.

6.1. Evaluation of the model on NSL-KDD and ISOT-CID Dataset

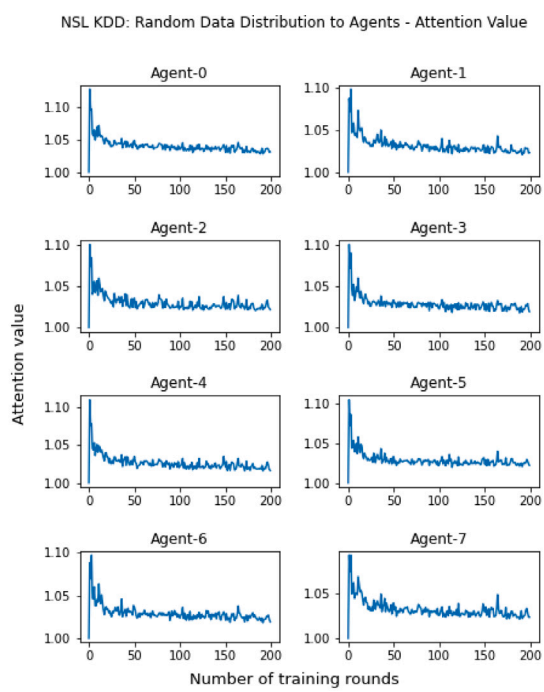
In this section, we evaluate the performance of our model on the NSL-KDD and the ISOT-CID datasets. For all the experiments, we plotted the accuracy of the model, loss obtained and the attention values for each of the agents after every round of training.

Fig. 3.a, .b, and .c show the plots corresponding to the NSL-KDD dataset with random distribution of the data to each of the agents. Similarly, Fig. 4.a, .b, and .c represent the plots corresponding to the random distribution experiment on the ISOT-CID Dataset. There are 8 subplots in each of the above figures. Figs. 3.a and 4.a corresponds to the graphs plotted against the accuracy and training round number for each of the eight agents. Figs. 3.b and 4.c show the plots for loss vs. round number. Similarly, Figs. 3.c and 4.c shows the plots against attention value of each of the agent vs. round number of the training process.

By observing the plots we can infer that the accuracy of the agents are increasing as the training process progresses while the loss of the agents are falling as the round number of the training increases. The attention values of the agents are also getting dropped with increase in the training round number. This is expected because of the fact that the attention value of an agent is dynamically determined by the model accuracy along with the number of training data samples in the current round. The attention value of the agent decreases as its accuracy increases since the number of training samples in each of the agents is almost equal for each round of the training process.

For the first experiment type, i.e., random data split, the parameters k and a have been tuned to the values 30 and 50 respectively, resulting in the best performance. The accuracy of the model is found to be 96.7% when it was trained on the NSL-KDD dataset with random distribution of data to the agents. The accuracy of the model on the ISOT-CID dataset is found to be 99.66%. For the random data distribution to the

(a) (b)



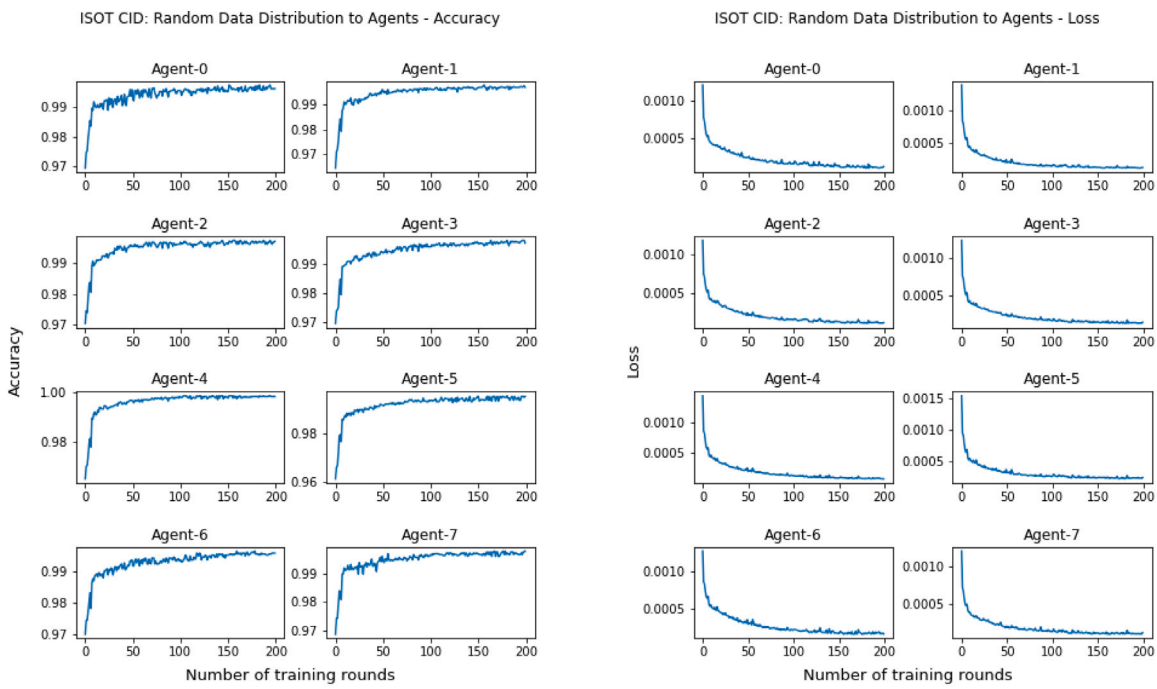
(c)

Fig. 3. Plots of NSL-KDD dataset with random data distribution.

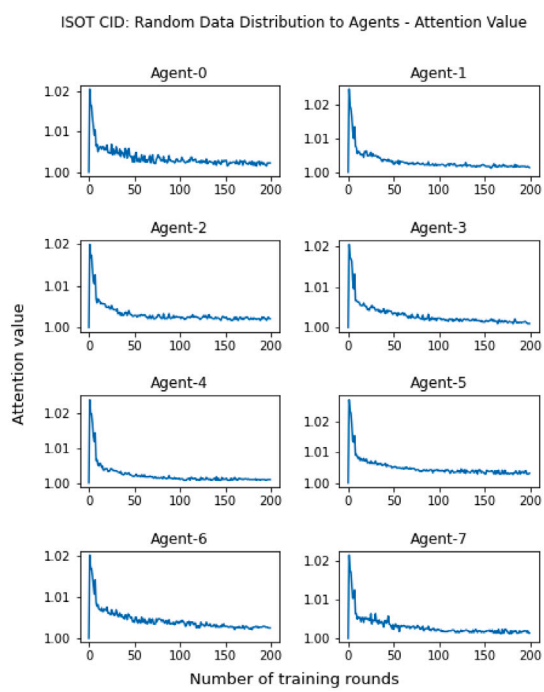
agents, the False Positive Rate (FPR) for the NSL-KDD is 0.019, whereas the FPR for the ISOT-CID dataset is 0.0017.

In second type experiments, data is distributed among the agents in a customized fashion. For the NSL-KDD dataset, one agent is supplied with data samples containing only the normal and DOS attack samples, while the other agent is given normal as well as Probe, U2R, and R2L attack types. We plotted the graphs for these experiments which are shown in Fig. 5. Similarly for the ISOT-CID dataset, each agent is supplied with the data which is collected on a separate day. The graphs for this experiment are shown in Fig. 6.

Figs. 5.a and 6.a depicts the plots of accuracy vs. training round number for the NSL-KDD and the ISOT-CID datasets respectively during the customized data experiment. Similarly the Figs. 5.b and 6.b are the graphs plotted against loss and the round number during training process. Figs. 5.c and 6.c shows plots against attention value vs. training round number for NSL-KDD and ISOT-CID respectively. In both the



(a) (b)



(c)

Fig. 4. Plots of ISOT-CID dataset with random data distribution.

plots, we can observe that the attention value drops as the training round number increases, which is expected because of increase in accuracy. Our system on customized data distribution gives similar performance like randomized data distribution. So it is verified that our system performs well also in customized data distribution.

During the experimentation of customized data split among the agents, the accuracy of the model on NSL-KDD dataset is found to be 96.09% and on the ISOT-CID dataset, it is 99.53%. The FPR on the models corresponding to NSL-KDD and ISOT-CID are 0.0017 and 0.0008 respectively. For the customized split experimentation, the attention multiplier parameters k and a are 50 000, 200 for NSL-KDD and 30, 50 for ISOT-CID datasets respectively.

Table 3 displays different performance metrics (such as accuracy, precision, FPR, and recall), for the two different types of experiments on both datasets. Fig. 7 shows ROC curves for the experiments conducted

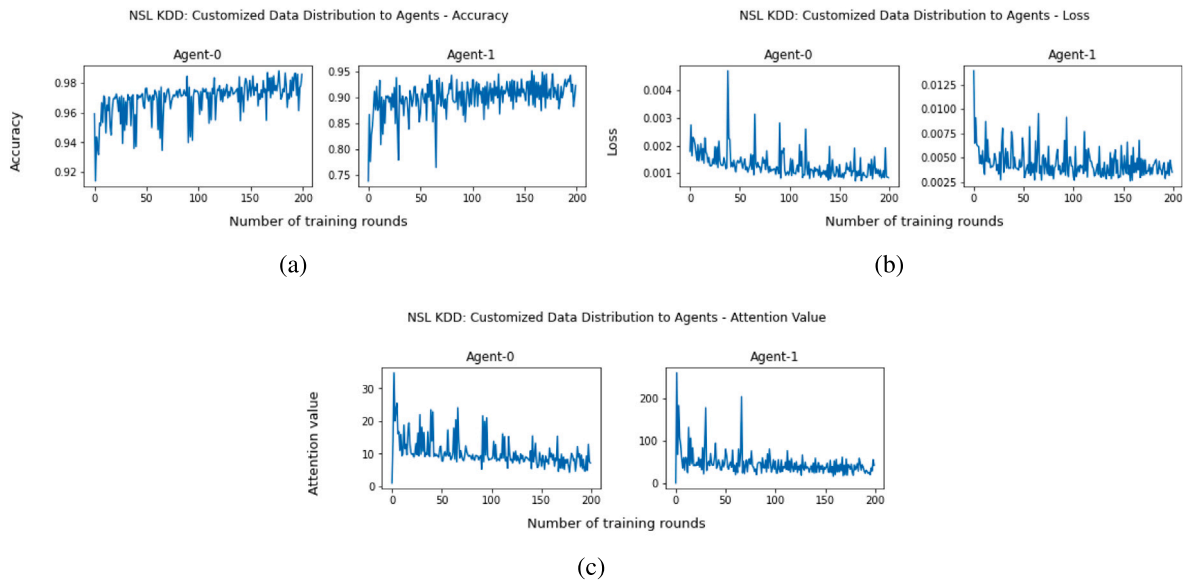


Fig. 5. Plots for of NSL-KDD dataset with customized data distribution.

Table 3
Evaluation of our model on the NSL-KDD and ISOT-CID datasets.

Data distribution type among agents	Dataset	Accuracy	FPR	Recall	Precision	F1-Score	AUC ROC
Random/Uniform	NSL KDD	0.9669	0.0195	0.9514	0.9769	0.964	0.9940
	ISOT CID	0.9966	0.0017	0.9468	0.9444	0.9456	0.9995
Customized	NSL KDD	0.9609	0.0105	0.9282	0.987	0.9567	0.9960
	ISOT CID	0.9953	0.0008	0.8762	0.9695	0.9205	0.9994

on both datasets. Area under the ROC curve (AUC) values for the ISOT-CID and NSL-KDD datasets are 0.994 and 0.996 respectively during the random or uniform data distribution among the agents. These values indicate show that, with random data distribution, the model can distinguish between malicious and non-malicious categories with 99.6% percent and 99.4% percent accuracy for the ISOT-CID and NSL-KDD datasets, respectively. Similarly, the AUC values for customized data distribution indicate that model can differentiate between attack and non-attack samples with 99.95% and 99.94% for NSL-KDD and ISOT-CID datasets respectively.

6.2. Impact of the proposed dynamic attention mechanism

In this section, we test the effectiveness of our proposed dynamic attention mechanism. The aim of this experiment is to know more about how the system benefits with the addition of dynamic attention values. Initially, the system is trained without using dynamic attention mechanism and we used the number of samples in the training as the weight during the model aggregation process. On the other hand, in the next experimental setup, we introduced the dynamic attention values into the system and observed the results. These experiments are carried out using the NSL-KDD dataset in addition to the ISOT-CID dataset.

For NSL-KDD dataset, the entire data is split into two parts, one part contains samples of DoS attack and the other parts contains samples of Probe, U2R and R2R attacks. Non-attack samples from the dataset are also distributed in the ratio of the number of attack samples in both the parts. Both the parts are allotted to two separate agents and we simulated the experiment. The training samples in the part containing DoS attacks are relatively much higher in magnitude compared to the count of training samples in the other part. For the ISOT-CID dataset, each agent is provided with the data collected on alternate days and the experiments are conducted with and without the dynamic attention mechanism. Table 4 shows the reported observations from these experiments. It indicates that by utilizing dynamic attention mechanism, we were able to improve the performance of the entire system when compared to the results against not utilizing dynamic attention mechanism.

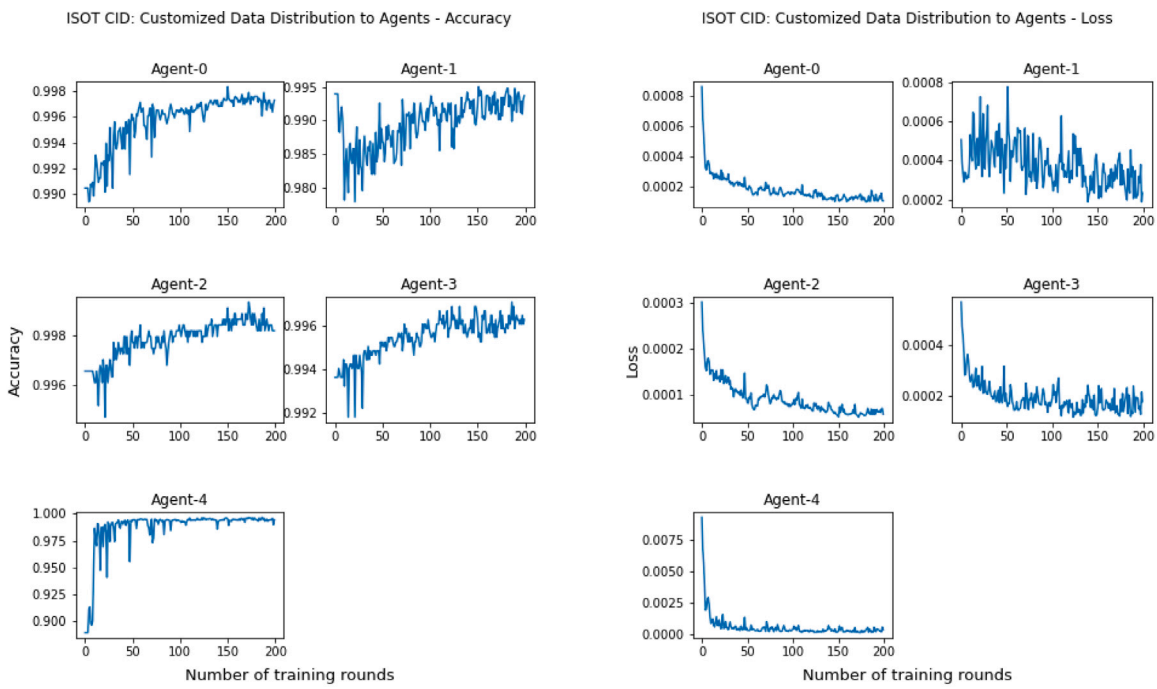
6.3. Performance of the system with increase in the size of network

In this section, we discuss about the impact of the network size on the overall performance of the system. In order to gain a better understanding of how the number of agents impacts the performance, we performed an experiment in which we simulated the system multiple times with varying number of agents in an increasing fashion During each simulation, we randomly divided the available data into approximately equal sized chunks to be distributed among the agents, and subsequently computed the overall average accuracy of the system. The experiment was done on both the datasets and the plots drawn on average accuracy vs. number of agents are shown in Fig. 8.

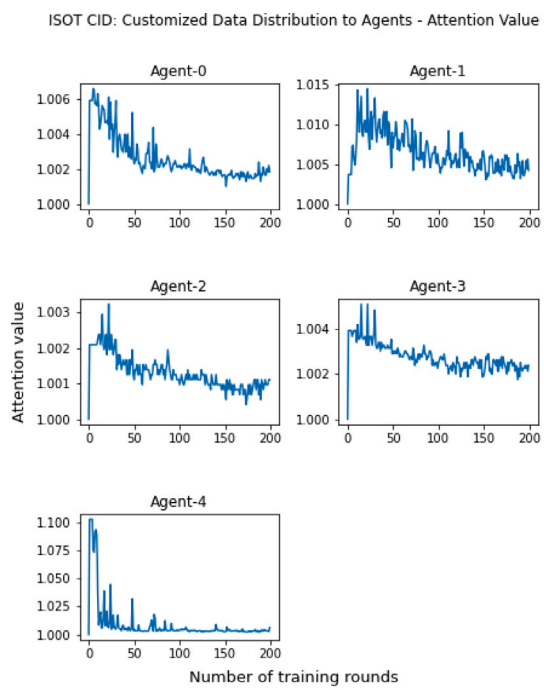
By observing the plots, we can interpret that the average accuracy of the system remains constant, when the number of agents increases, and the data is split into multiple distributed locations. As a result, the system can be easily scaled to large networks without affecting performance.

6.4. Comparison of our model with state of the art federated learning works

Federated Learning based Intrusion Detection Systems have been used in different contexts recently. Tian Dong et al. (2022) [53] proposed an FL-NIDS by combining the Federated Learning (FL) and interpretable Gradient Boosting Decision Tree (GBDT). Similarly, Chen et al. (2021) [54] proposed an FL-IDS for Wireless Sensor Networks (WSNs) that uses a cooperative aggregation algorithm. Burak Cetin et al. (2019) [55] proposed an Federated Learning Wireless Network Intrusion Detection System using Stacked Auto Encoder and tested on the AWID dataset. Li et al. (2020) [56] proposed an FL-NIDS for satellite–terrestrial integrated networks (STINs). Their approach outperformed



(a) (b)



(c)

Fig. 6. Plots of ISOT-CID dataset with customized data distribution.

traditional Deep Learning and Intrusion Detection Methods in terms of common performance metrics [54]. In Table 5, we have compared the performance of our model with existing state-of-the-art federated learning models. The existing works use federated learning and can be used for protecting data privacy, but none of them have addressed the case where the data distribution across the agents/nodes is uneven. However, our proposed model can be used in those cases as well and gives good results due to the dynamic attention value mechanism.

7. Limitations of the proposed solution

Apart from the various benefits of our proposed IDS architecture, there are few shortcomings as outlined below.

1. We derived the mathematical model to determine attention values dynamically based on the assumption that there would be a uniform data generation rate at each of the agents. We believe

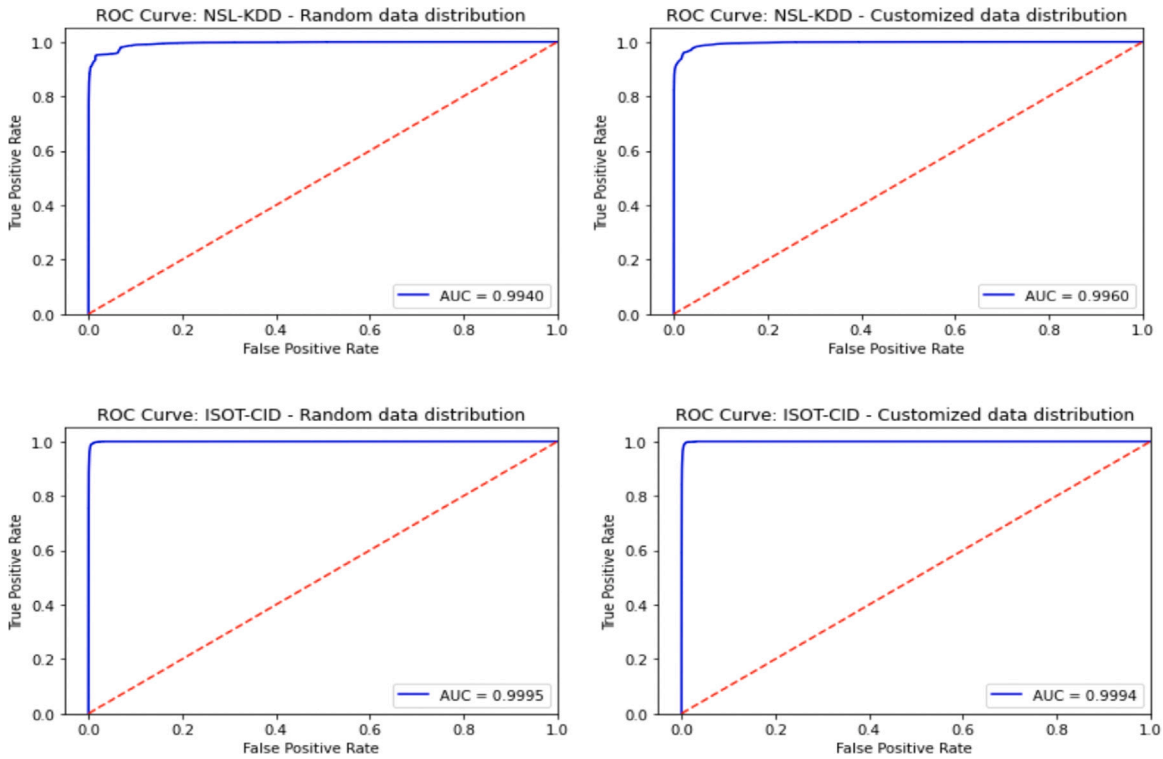


Fig. 7. ROC curves for the experiments on both the datasets.

Table 4
Comparison of accuracy with and without dynamic attention mechanism.

Dataset	Agent	Accuracy without dynamic attention	Accuracy with dynamic attention
NSL-KDD	A0	98.30%	98.50%
	A1	87.40%	92.30%
	weighted_avg	**95.95%**	**97.16%**
ISOT-CID	A0	98.70%	99.72%
	A1	98.45%	99.37%
	A2	98.65%	99.81%
	A3	98.61%	99.63%
	A4	98.27%	99.42%
	weighted_avg	**99.53%**	**99.60%**

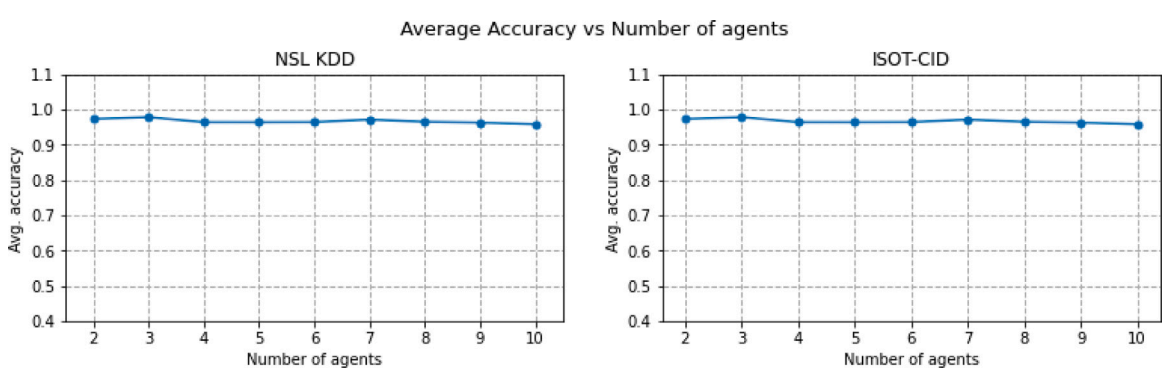


Fig. 8. Average accuracy vs. number of agents.

that this assumption would be valid in many networks as the average traffic flow at each of the agent nodes would remain uniform for a given period of time.

2. Despite the fact that our proposed methodology is based on the distributed architecture, it excludes the consideration of processing delay and communication overheads. We believe that the advancements in computation and low latency networks would help our proposed solution.

8. Conclusion and future work

In this paper, we proposed a Federated Deep Reinforcement Learning based IDS that uses dynamic attention mechanism to identify malicious network attacks. We implemented and evaluated our proposed model on the NSL-KDD and ISOT-CID datasets. To the best of our knowledge, this is the first work that uses the concept of federated learning along with the deep reinforcement learning approach and

Table 5
Comparison with state of the art federated learning algorithms.

Reference	Dataset	Accuracy	F1 Score	FPR	DAVC
[54]	CICIDS2017	99.28%	98.12%	0.73%	No
[53]	DDoS2019, MalDroid2020, Darknet2020, DoHBrw2020	89.63%	88.59%	–	No
[56]	Self Generated	88%	–	37%	No
[55]	AWID Dataset	89%	–	–	No
Our model	NSL-KDD, ISOT-CID	99.66%	96.4%	0.17%	Yes

DAVC: Dynamic Attention Value Mechanism.

prioritized experience replay. In addition, we proposed a mathematical model to dynamically determine the appropriate attention values for the agents resulting in improvement of the overall system performance. Further the experimental results discussed in Section 6 shows that our system is robust, capable of giving high accuracy along with low FPR and scalable to large networks as well. In future, we will extend our proposed IDS work to support non-uniform data generation rate at the agents. In addition to that, we plan to deploy and evaluate the proposed architecture on large scale real world networks.

CRediT authorship contribution statement

Sreekanth Vadigi: Conceptualization, Methodology, Validation, Writing – original draft. **Kamalakanta Sethi:** Conceptualization, Methodology, Writing – review & editing, Supervision. **Dinesh Mohanty:** Conceptualization, Methodology. **Shom Prasad Das:** Validation, Writing – review & editing. **Padmalochan Bera:** Supervision.

Declaration of competing interest

All authors have participated in (a) conception and design, or analysis and interpretation of the data; (b) drafting the article or revising it critically for important intellectual content; and (c) approval of the final version.

This manuscript has not been submitted to, nor is under review at, another journal or other publishing venue.

The authors have no affiliation with any organization with a direct or indirect financial interest in the subject matter discussed in the manuscript

The following authors have affiliations with organizations with direct or indirect financial interest in the subject matter discussed in the manuscript:

Sreekanth Vadigi, Indian Institute of Technology Bhubaneswar

Kamalakanta Sethi, Indian Institute of Information Technology SriCity

Dinesh Mohanty, Indian Institute of Technology Bhubaneswar

Shom Prasad Das, Birla Global University Bhubaneswar

Padmalochan Bera, Indian Institute of Technology Bhubaneswar

Data availability

No data was used for the research described in the article.

References

[1] Mbona I, Eloff JHP. Detecting zero-day intrusion attacks using semi-supervised machine learning approaches. IEEE Access 2022;10:69822–38. http://dx.doi.org/10.1109/ACCESS.2022.3187116.

[2] Li W, Yi P, Wu Y, Pan L, Li J. A new intrusion detection system based on KNN classification algorithm in wireless sensor network. J Electr Comput Eng 2014;2014:240217.

[3] Kuang F, Xu W, Zhang S. A novel hybrid KPCA and SVM with GA model for intrusion detection. Appl Soft Comput 2014;18:178–84.

[4] Reddy RR, Ramadevi Y, Sunitha KVN. Effective discriminant function for intrusion detection using SVM. In: Proc. int. conf. adv. comput. commun. inform. (ICACCI). 2016, p. 1148–53.

[5] Quinlan R. Induction of decision trees. Mach Learn 1986;1(1):81–106.

[6] Ross Quinlan J. C4.5: Programs for machine learning. Morgan Kaufmann; 1993.

[7] Bivens A, Palagiri C, Smith R, Szymanski B, Embrechts M. Network-based intrusion detection using neural networks. Intell Eng Syst Artif Neural Netw 2002;12(1):579–84.

[8] Lopez-Martin M, Carro B, Sanchez-Esguevillas A. Application of deep reinforcement learning to intrusion detection for supervised problems. Expert Syst Appl 141(2020). http://dx.doi.org/10.1016/j.eswa.2019.112963.

[9] Lavet VF, Henderson P, Islam R, Bellemare MG, Pineau J. An introduction to deep reinforcement learning. 2018, arXiv:1811.12560 [cs.LG].

[10] Nguyen Thanh Thi, Reddi Vijay Janapa. Deep reinforcement learning for cyber security. 2019, arXiv preprint arXiv:1906.05799.

[11] Sethi Kamalakanta, Venu Madhav Y, Kumar Rahul, Bera Padmalochan. Attention based multi-agent intrusion detection systems using reinforcement learning. J Inf Secur Appl 2021;61:102923. http://dx.doi.org/10.1016/j.jisa.2021.102923.

[12] Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan … G, Chintala S. Pytorch: An imperative style, high-performance deep learning library. Adv Neural Inf Process Syst 32.

[13] Isot cid website. [Online]. Available: https://www.uvic.ca/engineering/ece/isot/datasets/index.phpion.

[14] NSL-KDD dataset. [Online], Available: https://www.unb.ca/cic/datasets/nsl.html.

[15] Li Q, Wen Z, Wu Z, Hu S, Wang N, Li … Y, He B. A survey on federated learning systems: vision, hype and reality for data privacy and protection. IEEE Trans Knowl Data Eng.

[16] McMahan B, Moore E, Ramage D, Hampson S, y Arcas BA. Communication-efficient learning of deep networks from decentralized data. In: Artificial intelligence and statistics. PMLR; 2017, p. 1273–82.

[17] Otterlo Martijn, Wiering Marco. Reinforcement Learning and Markov Decision Processes. Reinforcement Learning: State of the Art. 3-42. http://dx.doi.org/10.1007/978-3-642-27645-3_1.

[18] Schaul T, Quan J, Antonoglou I, Silver D. Prioritized experience replay. 2016.

[19] Schaul Tom, et al. Prioritized experience replay. 2015, arXiv preprint arXiv:1511.05952.

[20] Huber PJ. Robust estimation of a location parameter. New York, NY: Springer New York; 1992, p. 492–518.

[21] Vaswani Ashish, et al. Attention is all you need. Adv Neural Inf Process Syst 2017;30.

[22] Luong Minh-Thang, Pham Hieu, Manning Christopher D. Effective approaches to attention-based neural machine translation. 2015, arXiv preprint arXiv:1508.04025v5.

[23] Bahdanau Dzmitry, Cho Kyunghyun, Bengio Yoshua. Neural machine translation by jointly learning to align and translate. 2015, arXiv preprint arXiv:1409.0473v7.

[24] Cheng Jianpeng, Dong Li, Lapata Mirella. Long short-term memory-networks for machine reading. 2015, arXiv preprint arXiv:1601.06733v7.

[25] Ingre B, Yadav A, Soni AK. Decision tree based intrusion detection system for NSL-KDD dataset. In: Satapathy A, editor. Information and communication technology for intelligent systems (ICTIS 2017) - Volume 2. ICTIS 2017. Smart innovation, systems and technologies, Vol. 84, Springer; http://dx.doi.org/10.1007/978-3-319-63645-0_23.

[26] Jing D, Chen H-B. SVM based network intrusion detection for the UNSW-NB15 dataset. In: 2019 IEEE 13th international conference on ASIC (ASICON). Chongqing, China; 2019, p. 1–4. http://dx.doi.org/10.1109/ASICON47005.2019.8983598.

[27] Meerja AJ, Ashu A, Rajani Kanth A. A Gaussian Naïve Bayes based intrusion detection system. In: Abraham A, Jabbar M, Tiwari S, Jesus I, editors. Proceedings of the 11th international conference on soft computing and pattern recognition (SoCPaR 2019). SoCPaR 2019. Advances in intelligent systems and computing, vol 1182, Cham: Springer, http://dx.doi.org/10.1007/978-3-030-49345-5_16.

[28] Pacheco J, Benitez VH, Félix-Herrán LC, Satam P. Artificial neural networks-based intrusion detection system for internet of things fog nodes. IEEE Access 2020;8:73907–18. http://dx.doi.org/10.1109/ACCESS.2020.2988055.
[29] Chen L, Kuang X, Xu A, Suo S, Yang Y. A novel network intrusion detection system based on CNN. In: 2020 eighth international conference on advanced cloud and big data (CBD). Taiyuan, China; 2020, p. 243–7. http://dx.doi.org/10.1109/CBD51900.2020.00051.
[30] Nayyar S, Arora S, Singh M. Recurrent neural network based intrusion detection system. In: 2020 international conference on communication and signal processing (ICCSP). Chennai, India; 2020, p. 0136–40. http://dx.doi.org/10.1109/ICCSP48568.2020.9182099.
[31] Alavizadeh H, Alavizadeh H, Jang-Jaccard J. Deep Q-learning based reinforcement learning approach for network intrusion detection. Computers 2022;11:41. http://dx.doi.org/10.3390/computers11030041.
[32] Jain K, Dubes RC. Algorithms for clustering data. Englewood Cliffs, NJ, USA: Prentice-Hall; 1988.
[33] Blowers M, Williams J. Machine learning applied to cyber operations. Network science and cybersecurity, New York, NY, USA: Springer; 2014, p. 55–175.
[34] Farnaaz N, Jabbar MA. Random forest modelling for network intrusion detection system. Procedia Comput Sci 2016;89:213217.
[35] Injadat M, Salo F, Nassif AB, Essex A, Shami A. Bayesian optimization with machine learning algorithms towards anomaly detection. In: 2018 IEEE global communications conference (GLOBECOM). 2018, p. 1–6. http://dx.doi.org/10.1109/GLOCOM.2018.8647714.
[36] Aljawarneh Shadi, Aldwairi Monther, Yassein Muneer Bani. Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. J Comput Sci 2018;25:152–60. http://dx.doi.org/10.1016/j.jocs.2017.03.006.
[37] Wang Z. Deep learning-based intrusion detection with adversaries. IEEE Access 2018;6:38367–84.
[38] Qureshi AH, Larijani H, Ahmad J, Mtetwa N. A heuristic intrusion detection system for internet-of-things (IoT). In: Arai K, Bhatia R, Kapoor S, editors. Intelligent computing. CompCom 2019. Advances in intelligent systems and computing, Vol. 997, Cham: Springer; 2019.
[39] Jiang F, et al. Deep learning based multi-channel intelligent attack detection for data security. IEEE Trans Sustain Comput 2018.
[40] Hassan Mohammad Mehedi, Gumaei Abdu, Alsanad Ahmed, Alrubaian Majed, Fortino Giancarlo. A hybrid deep learning model for efficient intrusion detection in big data environment. Inform Sci 2020;513:386–96. http://dx.doi.org/10.1016/j.ins.2019.10.069.
[41] Wang Zhendong, Liu Yaodi, He Daojing, Chan Sammy. Intrusion detection methods based on integrated deep learning model. Comput Secur 2021;103:102177. http://dx.doi.org/10.1016/j.cose.2021.102177.
[42] Cannady J. Next generation intrusion detection: Autonomous reinforcement learning of network attacks. In: Proceedings of the 23rd national information systems security conference. Baltimore; 2000, p. 1–12.
[43] Servin A, Kudenko D. Multi-agent reinforcement learning for intrusion detection. In: Proceedings of the 5th, 6th and 7th European conference on adaptive and learning agents and multi-agent systems: adaptation and multi-agent learning. 2008, p. 211–23.
[44] Xu X, Xie T. A reinforcement learning approach for host-based intrusion detection using sequences of system calls. In: Proceedings of international conference on intelligent computing. Lecture notes in computer science, LNCS, Vol. 3644, 2005, p. 995–1003.
[45] Konecny J, McMahan HB, Ramage D. Federated Optimization: Distributed Optimization Beyond the Datacenter. ArXiv, abs/1511.03575.
[46] Agrawal S, Sarkar S, Aouedi O, Yenduri G, Piamrat K, Bhattacharya S, Maddikunta PK, Gadekallu TR. Federated Learning for Intrusion Detection System: Concepts, Challenges and Future Directions. ArXiv, abs/2106.09527.
[47] Zhuo HH, Feng W, Lin Y, Xu Q, Yang Q. Federated deep reinforcement learning. arXiv preprint arXiv:1901.08277.
[48] Wang X, Wang C, Li X, Leung VCM, Taleb T. Federated deep reinforcement learning for internet of things with decentralized cooperative edge caching. IEEE Internet Things J 2020;7(10):9441–55. http://dx.doi.org/10.1109/JIOT.2020.2986803.
[49] Wang X, Li R, Wang C, Li X, Taleb T, Leung VCM. Attention-weighted federated deep reinforcement learning for device-to-device assisted heterogeneous collaborative edge caching. IEEE J Sel Areas Commun 2021;39(1):154–69. http://dx.doi.org/10.1109/JSAC.2020.3036946.
[50] Meena G, Choudhary RR. A review paper on IDS classification using KDD 99 and NSL KDD dataset in WEKA. In: 2017 international conference on computer, communications and electronics (Comptelix). 2017, p. 553–8. http://dx.doi.org/10.1109/COMPTELIX.2017.8004032.
[51] Tavallaee M, Bagheri E, Lu W, Ghorbani A. A detailed analysis of the KDD CUP 99 data set. In: 2nd IEEE symposium on computational intelligence for security and defense applications (CISDA). 2009.
[52] Aldribi A, Traoré I, Moa B, Nwamuo O. Hypervisor-based cloud intrusion detection through online multivariate statistical change tracking. Comput Secur 88.
[53] Dong T, Li S, Qiu H, Lu J. An Interpretable Federated Learning-based Network Intrusion Detection Framework.
[54] Chen Z, Lv N, Liu P, Fang Y, Chen K, Pan W. Intrusion detection for wireless edge networks based on federated learning. IEEE Access 2020;8:217463–72. http://dx.doi.org/10.1109/ACCESS.2020.3041793.
[55] Cetin B, Lazar A, Kim J, Sim A, Wu K. Federated wireless network intrusion detection. In: 2019 IEEE international conference on big data (Big Data). Los Angeles, CA, USA; 2019, p. 6004–6. http://dx.doi.org/10.1109/BigData47090.2019.9005507.
[56] Li K, Zhou H, Tu Z, Wang W, Zhang H. Distributed network intrusion detection system in satellite-terrestrial integrated networks using federated learning. IEEE Access 2020;8:214852–65. http://dx.doi.org/10.1109/ACCESS.2020.3041641.